Operating Systems (CSD204): Graded Lab Assignment 1

Submitting this Assignment:

You will submit (upload) this assignment on Blackboard. Email submissions will not be accepted. Make sure that you upload the correct and complete file for the given assignment.

1. The following two files are required to be uploaded on Blackboard:

- One Java file containing the complete code.
  - ✓ Project Name in Netbeans: **osgla1**
  - ✓ Class Name: **cpuscheduling**
  - ✓ The java file must contain your name and roll no (as comments).
- Pdf - Paste your Java code after the given question in this document. Also, paste the screenshots of input screen and output screen.
  Name this document as **GLA1_OS2019_John_Doe.pdf** in case your name is John Doe [You are required to strictly follow the naming convention.]

2. Assignment submitted after due date and time will not be evaluated and a score of zero will be awarded for this assignment.

3. Materials copied from the Internet or otherwise will attract penalty as per the course policy.

Grading: This assignment has 10 points (with **weightage of 10%** in your overall 100 points).

Due Date: **11:55 pm, March 9, 2019**

Question

1. Implement the following CPU scheduling algorithms in Java:

a. FCFS
b. Preemptive SJF
c. Non-preemptive SJF
d. Preemptive Priority (lower the integer, higher the priority)
e. Non-Preemptive Priority (lower the integer, higher the priority)
f. Round Robin

The input screen (template discussed in the class) must contain no. of processes (<= 6), arrival time, burst time, priority, time quantum (for RR), checkboxes for algorithm selection and button for computation.

The output screen (template discussed in the class) must contain the Gantt chart, waiting time and turnaround time for each process, average waiting time, and average turnaround time.

Operating Systems (CSD204): Graded Lab Assignment 1

Java Code

//paste here

```
/*
Made by Kabir Kanha Arora
08-Mar-2019
Shiv Nadar University
1710110165
CSE-2021
*/

import jdk.swing.interop.SwingInterOpUtils;

import javax.swing.*;
import javax.swing.text.DefaultEditorKit;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.FocusEvent;
import java.awt.event.FocusListener;
import java.util.ArrayList;

public class cpuscheduling extends JFrame {

    //Array of strings for selecting number of processes
    private String[] arr = {"1", "2", "3", "4", "5", "6"};

    //Combo Box for selection of number of proceses
    private JComboBox cb_choice = new JComboBox(arr);

    //Creating JLabels
    private JLabel label_pno = new JLabel("Number of processes (<=6)");
    private JLabel label_bt = new JLabel("Burst Times *");
    private JLabel label_at = new JLabel("Arrival Times *");
    private JLabel label_pri = new JLabel("Priorities");
    private JLabel label_tq = new JLabel("Time Quantum");
    private JLabel label_algo = new JLabel("         Scheduling Algorithm");
    private JLabel disclaimer = new JLabel("(Please enter integral values only)");

    //Creating JTextFields
    private JTextField jtf_bt = new JTextField("Enter comma separated burst times");
    private JTextField jtf_at = new JTextField("Enter comma separated arrival times");
    private JTextField jtf_pri = new JTextField("Enter comma separated priorities (0 =
Highest)");
    private JTextField jtf_tq = new JTextField("Enter time quantum in milliseconds");

    //Creating checkboxes for CPU scheduling algorithms
    private JCheckBox jcb_FCFS = new JCheckBox("FCFS");
    private JCheckBox jcb_RR = new JCheckBox("Round Robin");
    private JCheckBox jcb_preSJF = new JCheckBox("Preemptive SJF");
    private JCheckBox jcb_npSJF = new JCheckBox("Non-Preemptive SJF");
    private JCheckBox jcb_prePri = new JCheckBox("Preemptive Priority");
    private JCheckBox jcb_npPri = new JCheckBox("Non-Preemptive Priority");

    //Declaring boolean variable to track checkbox status
    private static boolean b_FCFS = false;
    private static boolean b_RR = false;
    private static boolean b_preSJF = false;
    private static boolean b_npSJF = false;
    private static boolean b_prePri = false;
    private static boolean b_npPri = false;

    //JButton to compute
    private JButton jbt_compute = new JButton("COMPUTE");

    //Another JFrame for Gantt charts
    private JFrame ganttframe = new JFrame();

    //Stores number of processes
    private static int cnt_process;

    //Array lists
```

```
    private static ArrayList<Integer> arr_at = new ArrayList<>();
    private static ArrayList<Integer> arr_bt = new ArrayList<>();
    private static ArrayList<Integer> arr_pri = new ArrayList<>();
    private static ArrayList<String> parr = new ArrayList<>();
    private static ArrayList<String> parrcopy = new ArrayList<>();

    //FCFS
    private static ArrayList<Integer> arr_st_fcfs = new ArrayList<>();
    private static ArrayList<Integer> arr_et_fcfs = new ArrayList<>();
    private static ArrayList<Integer> arr_wt_fcfs = new ArrayList<>();
    private static ArrayList<Integer> arr_tat_fcfs = new ArrayList<>();
    private static ArrayList<Integer> arr_waitstart_fcfs = new ArrayList<>();
    private static ArrayList<Integer> arr_waitend_fcfs = new ArrayList<>();
    private static double avg_wt_fcfs = 0;
    private static double avg_tat_fcfs = 0;
    private static int tt_fcfs = 0;

    //NP SJF
    private static ArrayList<Integer> arr_st_npsjf = new ArrayList<>();
    private static ArrayList<Integer> arr_et_npsjf = new ArrayList<>();
    private static ArrayList<Integer> arr_wt_npsjf = new ArrayList<>();
    private static ArrayList<Integer> arr_tat_npsjf = new ArrayList<>();
    private static ArrayList<Integer> arr_waitstart_npsjf = new ArrayList<>();
    private static ArrayList<Integer> arr_waitend_npsjf = new ArrayList<>();
    private static ArrayList<Integer> arr_at_npsjf = new ArrayList<>();
    private static ArrayList<String> parr_npsjf = new ArrayList<>();
    private static double avg_wt_npsjf = 0;
    private static double avg_tat_npsjf = 0;
    private static int tt_npsjf = 0;

    //Pre SJF
    private static ArrayList<Integer> arr_st_preSJF = new ArrayList<>();
    private static ArrayList<Integer> arr_et_preSJF = new ArrayList<>();
    private static ArrayList<Integer> arr_wt_preSJF = new ArrayList<>();
    private static ArrayList<Integer> arr_tat_preSJF = new ArrayList<>();
    private static ArrayList<Integer> arr_waitstart_preSJF = new ArrayList<>();
    private static ArrayList<Integer> arr_waitend_preSJF = new ArrayList<>();
    private static ArrayList<Integer> arr_timeleft_preSJF = new ArrayList<>();
    private static ArrayList<String> parr_preSJF = new ArrayList<>();
    private static double avg_wt_preSJF = 0;
    private static double avg_tat_preSJF = 0;
    private static int tt_preSJF = 0;

    //Pre Priority
    private static ArrayList<Integer> arr_st_prePri = new ArrayList<>();
    private static ArrayList<Integer> arr_et_prePri = new ArrayList<>();
    private static ArrayList<Integer> arr_wt_prePri = new ArrayList<>();
    private static ArrayList<Integer> arr_tat_prePri = new ArrayList<>();
    private static ArrayList<Integer> arr_waitstart_prePri = new ArrayList<>();
    private static ArrayList<Integer> arr_waitend_prePri = new ArrayList<>();
    private static ArrayList<Integer> arr_timeleft_prePri = new ArrayList<>();
    private static ArrayList<String> parr_prePri = new ArrayList<>();
    private static double avg_wt_prePri = 0;
    private static double avg_tat_prePri = 0;
    private static int tt_prePri = 0;

    //NP Priority
    private static ArrayList<Integer> arr_st_npPri = new ArrayList<>();
    private static ArrayList<Integer> arr_et_npPri = new ArrayList<>();
    private static ArrayList<Integer> arr_wt_npPri = new ArrayList<>();
    private static ArrayList<Integer> arr_tat_npPri = new ArrayList<>();
    private static ArrayList<Integer> arr_waitstart_npPri = new ArrayList<>();
    private static ArrayList<Integer> arr_waitend_npPri = new ArrayList<>();
    private static ArrayList<Integer> arr_at_npPri = new ArrayList<>();
    private static ArrayList<String> parr_npPri = new ArrayList<>();
    private static double avg_wt_npPri = 0;
    private static double avg_tat_npPri = 0;
    private static int tt_npPri = 0;

    //RR
    private static int tq = 0;
    private static ArrayList<Integer> arr_st_RR = new ArrayList<>();
    private static ArrayList<Integer> arr_et_RR = new ArrayList<>();
    private static ArrayList<Integer> arr_wt_RR = new ArrayList<>();
    private static ArrayList<Integer> arr_tat_RR = new ArrayList<>();
```

```
    private static ArrayList<Integer> arr_waitstart_RR = new ArrayList<>();
    private static ArrayList<Integer> arr_waitend_RR = new ArrayList<>();
    private static ArrayList<Integer> arr_timeleft_RR = new ArrayList<>();
    private static ArrayList<String> parr_RR = new ArrayList<>();
    private static double avg_wt_RR = 0;
    private static double avg_tat_RR = 0;
    private static int tt_RR = 0;
    private static int N = 0;

    //Colour scheme
    static int[] red = {113, 165, 224, 223, 38, 35, 255};
    static int[] green = {45, 24, 153, 234, 155, 32, 255};
    static int[] blue = {150, 31, 83, 70, 48, 229, 255};

    cpuscheduling() {
        jtf_at.addFocusListener(new FocusListener() {
            @Override
            public void focusGained(FocusEvent e) {
                if (jtf_at.getText().equals("Enter comma separated arrival times"))
                    jtf_at.setText("");
            }

            @Override
            public void focusLost(FocusEvent e) {
                if (jtf_at.getText().equals(""))
                    jtf_at.setText("Enter comma separated arrival times");
            }
        });

        jtf_bt.addFocusListener(new FocusListener() {
            @Override
            public void focusGained(FocusEvent e) {
                if (jtf_bt.getText().equals("Enter comma separated burst times"))
                    jtf_bt.setText("");
            }

            @Override
            public void focusLost(FocusEvent e) {
                if (jtf_bt.getText().equals(""))
                    jtf_bt.setText("Enter comma separated burst times");
            }
        });

        jtf_pri.addFocusListener(new FocusListener() {
            @Override
            public void focusGained(FocusEvent e) {
                if (jtf_pri.getText().equals("Enter comma separated priorities (0 = Highest)"))
                    jtf_pri.setText("");
            }

            @Override
            public void focusLost(FocusEvent e) {
                if (jtf_pri.getText().equals(""))
                    jtf_pri.setText("Enter comma separated priorities (0 = Highest)");
            }
        });

        jtf_tq.addFocusListener(new FocusListener() {
            @Override
            public void focusGained(FocusEvent e) {
                if (jtf_tq.getText().equals("Enter time quantum in milliseconds"))
                    jtf_tq.setText("");
            }

            @Override
            public void focusLost(FocusEvent e) {
                if (jtf_tq.getText().equals(""))
                    jtf_tq.setText("Enter time quantum in milliseconds");
            }
        });

        //Setting layouts of all the panels
        JPanel panel_main = new JPanel();
        panel_main.setLayout(new BoxLayout(panel_main, BoxLayout.PAGE_AXIS));
        JPanel panel_pno = new JPanel();
```

```
        panel_pno.setLayout(new BoxLayout(panel_pno, BoxLayout.LINE_AXIS));
        JPanel panel_bt = new JPanel();
        panel_bt.setLayout(new BoxLayout(panel_bt, BoxLayout.LINE_AXIS));
        JPanel panel_at = new JPanel();
        panel_at.setLayout(new BoxLayout(panel_at, BoxLayout.LINE_AXIS));
        JPanel panel_pri = new JPanel();
        panel_pri.setLayout(new BoxLayout(panel_pri, BoxLayout.LINE_AXIS));
        JPanel panel_tq = new JPanel();
        panel_tq.setLayout(new BoxLayout(panel_tq, BoxLayout.LINE_AXIS));
        JPanel panel_cb = new JPanel();
        panel_cb.setLayout(new GridLayout(3, 3));

        //Adding components to their respective panels
        panel_pno.add(Box.createRigidArea(new Dimension(30, 0)));
        panel_pno.add(label_pno);
        panel_pno.add(cb_choice);
        panel_pno.add(Box.createRigidArea(new Dimension(30, 0)));
        panel_bt.add(label_bt);
        panel_bt.add(jtf_bt);
        panel_at.add(label_at);
        panel_at.add(jtf_at);
        panel_pri.add(label_pri);
        panel_pri.add(jtf_pri);
        panel_tq.add(label_tq);
        panel_tq.add(jtf_tq);
        panel_cb.add(Box.createRigidArea(new Dimension(30, 0)));
        panel_cb.add(jcb_FCFS);
        panel_cb.add(jcb_RR);
        panel_cb.add(label_algo);
        panel_cb.add(jcb_preSJF);
        panel_cb.add(jcb_npSJF);
        panel_cb.add(Box.createRigidArea(new Dimension(30, 0)));
        panel_cb.add(jcb_prePri);
        panel_cb.add(jcb_npPri);

        //Setting sizes for the labels and textfields/Combo box
        label_pno.setMaximumSize(new Dimension(200, 20));
        label_pno.setPreferredSize(new Dimension(200, 20));
        cb_choice.setMaximumSize(new Dimension(300, 20));
        cb_choice.setPreferredSize(new Dimension(300, 20));
        label_bt.setMaximumSize(new Dimension(200, 20));
        label_bt.setPreferredSize(new Dimension(200, 20));
        jtf_bt.setMaximumSize(new Dimension(300, 20));
        jtf_bt.setPreferredSize(new Dimension(300, 20));
        label_at.setMaximumSize(new Dimension(200, 20));
        label_at.setPreferredSize(new Dimension(200, 20));
        jtf_at.setMaximumSize(new Dimension(300, 20));
        jtf_at.setPreferredSize(new Dimension(300, 20));
        label_pri.setMaximumSize(new Dimension(200, 20));
        label_pri.setPreferredSize(new Dimension(200, 20));
        jtf_pri.setMaximumSize(new Dimension(300, 20));
        jtf_pri.setPreferredSize(new Dimension(300, 20));
        label_tq.setMaximumSize(new Dimension(200, 20));
        label_tq.setPreferredSize(new Dimension(200, 20));
        jtf_tq.setMaximumSize(new Dimension(300, 20));
        jtf_tq.setPreferredSize(new Dimension(300, 20));
        jbt_compute.setAlignmentX(CENTER_ALIGNMENT);
        disclaimer.setAlignmentX(CENTER_ALIGNMENT);

        //Setting fonts for the components
        jtf_bt.setFont(new Font("Arial", Font.ITALIC, 11));
        jtf_at.setFont(new Font("Arial", Font.ITALIC, 11));
        jtf_pri.setFont(new Font("Arial", Font.ITALIC, 11));
        jtf_tq.setFont(new Font("Arial", Font.ITALIC, 11));
        disclaimer.setFont(new Font("Arial", Font.ITALIC, 9));
        jtf_pri.setEnabled(false);
        jtf_tq.setEnabled(false);

        //Adding panels into the main panel
        panel_main.add(Box.createRigidArea(new Dimension(0, 10)));
        panel_main.add(panel_pno);
        panel_main.add(Box.createRigidArea(new Dimension(0, 20)));
        panel_main.add(panel_bt);
        panel_main.add(Box.createRigidArea(new Dimension(0, 2)));
        panel_main.add(panel_at);
```

```
        panel_main.add(Box.createRigidArea(new Dimension(0, 15)));
        panel_main.add(panel_cb);
        panel_main.add(Box.createRigidArea(new Dimension(0, 15)));
        panel_main.add(panel_pri);
        panel_main.add(Box.createRigidArea(new Dimension(0, 2)));
        panel_main.add(panel_tq);
        panel_main.add(Box.createRigidArea(new Dimension(0, 20)));
        panel_main.add(disclaimer);
        panel_main.add(Box.createRigidArea(new Dimension(0, 10)));
        panel_main.add(jbt_compute);
        panel_main.add(Box.createRigidArea(new Dimension(0, 10)));
        this.add(panel_main);

        //Adding actions listeners to checkboxes
        jcb_FCFS.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                b_FCFS = b_FCFS != true;
            }
        });

        jcb_RR.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (b_RR == true) {
                    b_RR = false;
                    jtf_tq.setEnabled(false);
                } else {
                    b_RR = true;
                    jtf_tq.setEnabled(true);
                }
            }
        });

        jcb_preSJF.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                b_preSJF = !b_preSJF;
            }
        });

        jcb_npSJF.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                b_npSJF = b_npSJF != true;
            }
        });

        jcb_prePri.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (b_prePri) {
                    b_prePri = false;
                    if (!b_npPri)
                        jtf_pri.setEnabled(false);
                } else {
                    b_prePri = true;
                    jtf_pri.setEnabled(true);
                }
            }
        });

        jcb_npPri.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (b_npPri) {
                    b_npPri = false;
                    if (!b_prePri)
                        jtf_pri.setEnabled(false);
                } else {
                    b_npPri = true;
                    jtf_pri.setEnabled(true);
                }
            }
        });
```

```
jbt_compute.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        //Clearing all array lists
        //FCFS
        arr_bt.removeAll(arr_bt);
        arr_at.removeAll(arr_at);
        arr_pri.removeAll(arr_pri);
        arr_wt_fcfs.removeAll(arr_wt_fcfs);
        arr_tat_fcfs.removeAll(arr_tat_fcfs);
        arr_et_fcfs.removeAll(arr_et_fcfs);
        arr_st_fcfs.removeAll(arr_st_fcfs);
        parr.removeAll(parr);

        //Pre SJF
        arr_waitend_preSJF.removeAll(arr_waitend_preSJF);
        arr_waitstart_preSJF.removeAll(arr_waitstart_preSJF);
        arr_st_preSJF.removeAll(arr_st_preSJF);
        arr_et_preSJF.removeAll(arr_et_preSJF);
        arr_wt_preSJF.removeAll(arr_wt_preSJF);
        arr_tat_preSJF.removeAll(arr_tat_preSJF);
        arr_timeleft_preSJF.removeAll(arr_timeleft_preSJF);
        parr_preSJF.removeAll(parr_preSJF);

        //NP SJF
        arr_waitend_npsjf.removeAll(arr_waitend_npsjf);
        arr_waitstart_npsjf.removeAll(arr_waitstart_npsjf);
        arr_st_npsjf.removeAll(arr_st_npsjf);
        arr_et_npsjf.removeAll(arr_et_npsjf);
        arr_wt_npsjf.removeAll(arr_wt_npsjf);
        arr_tat_npsjf.removeAll(arr_tat_npsjf);
        arr_at_npsjf.removeAll(arr_at_npsjf);
        parr_npsjf.removeAll(parr_npsjf);

        //Pre Priority
        arr_waitend_prePri.removeAll(arr_waitend_prePri);
        arr_waitstart_prePri.removeAll(arr_waitstart_prePri);
        arr_st_prePri.removeAll(arr_st_prePri);
        arr_et_prePri.removeAll(arr_et_prePri);
        arr_wt_prePri.removeAll(arr_wt_prePri);
        arr_tat_prePri.removeAll(arr_tat_prePri);
        arr_timeleft_prePri.removeAll(arr_timeleft_prePri);
        parr_prePri.removeAll(parr_prePri);

        //NP Priority
        arr_waitend_npPri.removeAll(arr_waitend_npPri);
        arr_waitstart_npPri.removeAll(arr_waitstart_npPri);
        arr_st_npPri.removeAll(arr_st_npPri);
        arr_et_npPri.removeAll(arr_et_npPri);
        arr_wt_npPri.removeAll(arr_wt_npPri);
        arr_tat_npPri.removeAll(arr_tat_npPri);
        arr_at_npPri.removeAll(arr_at_npPri);
        parr_npPri.removeAll(parr_npPri);

        //Round Robin
        arr_waitend_RR.removeAll(arr_waitend_RR);
        arr_waitstart_RR.removeAll(arr_waitstart_RR);
        arr_st_RR.removeAll(arr_st_RR);
        arr_et_RR.removeAll(arr_et_RR);
        arr_wt_RR.removeAll(arr_wt_RR);
        arr_tat_RR.removeAll(arr_tat_RR);
        arr_timeleft_RR.removeAll(arr_timeleft_RR);
        parr_RR.removeAll(parr_RR);

        //Creating graphics panel
        JPanel canvas = new Canvas();

        //Getting number of processes
        cnt_process = cb_choice.getSelectedIndex() + 1;
        //Making an array of processes
        for (int i = 0; i < cnt_process; ++i) {
            parr.add("P" + (i + 1));
        }
        //Flag for checking if any conditions have been violated
```

```
                int flag = 0;

                //Getting burst times into an array
                if (!(jtf_bt.getText().equals("") || jtf_bt.getText().equals("Enter comma
separated burst times"))) {
                    String[] sarr_bt = jtf_bt.getText().replaceAll("\\s", "").split(",", -1);
                    if (sarr_bt.length != cnt_process) {
                        JOptionPane.showMessageDialog(null, "Number of entries in burst times do
not match number of processes.");
                        flag = -1;
                    } else {
                        for (int i = 0; i < sarr_bt.length; ++i) {
                            try {
                                arr_bt.add(i, Integer.parseInt(sarr_bt[i]));
                            } catch (Exception ex) {
                                JOptionPane.showMessageDialog(null, "Invalid values in burst
times.");
                                flag = -1;
                            }
                        }
                    }
                } else {
                    JOptionPane.showMessageDialog(null, "Number of entries in burst times do not
match number of processes.");
                    flag = -1;
                }
                //Getting arrival times into an array
                if (!(jtf_at.getText().equals("") || jtf_at.getText().equals("Enter comma
separated arrival times"))) {
                    String[] sarr_at = jtf_at.getText().replaceAll("\\s", "").split(",", -1);
                    if (sarr_at.length != cnt_process) {
                        JOptionPane.showMessageDialog(null, "Number of entries in arrival times
do not match number of processes.");
                        flag = -1;
                    } else {
                        for (int i = 0; i < sarr_at.length; ++i) {
                            try {
                                arr_at.add(i, Integer.parseInt(sarr_at[i]));
                            } catch (Exception ex) {
                                JOptionPane.showMessageDialog(null, "Invalid values in arrival
times.");
                                flag = -1;
                            }
                        }
                    }
                } else {
                    JOptionPane.showMessageDialog(null, "Number of entries in arrival times do
not match number of processes.");
                    flag = -1;
                }
                //Getting priorities into an array
                if (b_prePri || b_npPri) {
                    if (!(jtf_pri.getText().equals("") || jtf_pri.getText().equals("Enter comma
separated priorities (0 = Highest)"))) {
                        String[] sarr_pri = jtf_pri.getText().replaceAll("\\s", "").split(",", -
1);
                        if (sarr_pri.length != cnt_process) {
                            JOptionPane.showMessageDialog(null, "Number of entries in priorities
do not match number of processes.");
                            flag = -1;
                        } else {
                            for (int i = 0; i < sarr_pri.length; ++i) {
                                try {
                                    arr_pri.add(i, Integer.parseInt(sarr_pri[i]));
                                } catch (Exception ex) {
                                    JOptionPane.showMessageDialog(null, "Invalid values in
priorities.");
                                    flag = -1;
                                }
                            }
                        }
                    } else {
                        JOptionPane.showMessageDialog(null, "Number of entries in priorities do
not match number of processes.");
                        flag = -1;
```

```
                    }
                }

                //Getting time quantum in an integer variable
                if (b_RR) {
                    if (!(jtf_tq.getText().equals("") || jtf_tq.getText().equals("Enter time
quantum in milliseconds"))) {
                        try {
                            tq = Integer.parseInt(jtf_tq.getText());
                        } catch (Exception ex) {
                            JOptionPane.showMessageDialog(null, "Please correct time quantum");
                            flag = -1;
                        }
                    } else {
                        JOptionPane.showMessageDialog(null, "Please correct time quantum.");
                        flag = -1;
                    }
                }

                for (int i = 0; i < arr_at.size() - 1; ++i)
                    for (int j = 0; j < arr_at.size() - i - 1; ++j)
                        if (arr_at.get(j) > arr_at.get(j + 1)) {
                            int temp1 = arr_at.get(j);
                            arr_at.set(j, arr_at.get(j + 1));
                            arr_at.set(j + 1, temp1);
                            String temp_s = parr.get(j);
                            parr.set(j, parr.get(j + 1));
                            parr.set(j + 1, temp_s);
                            int temp2 = arr_bt.get(j);
                            arr_bt.set(j, arr_bt.get(j + 1));
                            arr_bt.set(j + 1, temp2);
                        }

                if (b_FCFS && flag != -1) {
                    int j = 0;
                    int delay = 0;
                    int tot_wt = 0;
                    int tot_tat = 0;
                    for (int i = 0; i < arr_at.size(); ++i) {
                        if (delay < arr_at.get(i)) {
                            arr_waitstart_fcfs.add(j, delay);
                            delay = arr_at.get(i);
                            arr_waitend_fcfs.add(j, delay);
                            j++;
                        }
                        arr_st_fcfs.add(i, delay);
                        arr_et_fcfs.add(i, arr_st_fcfs.get(i) + arr_bt.get(i));
                        arr_tat_fcfs.add(i, arr_et_fcfs.get(i) - arr_at.get(i));
                        tot_tat += arr_tat_fcfs.get(i);
                        arr_wt_fcfs.add(i, arr_tat_fcfs.get(i) - arr_bt.get(i));
                        tot_wt += arr_wt_fcfs.get(i);
                        delay += arr_bt.get(i);
                    }
                    avg_tat_fcfs = (tot_tat * 1.0) / cnt_process;
                    avg_wt_fcfs = (tot_wt * 1.0) / cnt_process;
                    tt_fcfs = delay;
                }

                if (b_npSJF && flag != -1) {
                    boolean[] arr_av_npSJF = new boolean[cnt_process];
                    boolean[] arr_status_npSJF = new boolean[cnt_process];
                    boolean flag_npSJF;
                    for (int i = 0; arr_st_npsjf.size() < cnt_process; ) {
                        flag_npSJF = false;
                        for (int j = 0; j < arr_at.size(); ++j) {
                            if (i >= arr_at.get(j))
                                arr_av_npSJF[j] = true;
                        }
                        int min = Integer.MAX_VALUE;
                        int min_index = -1;
                        for (int j = 0; j < cnt_process; ++j) {
                            if (arr_av_npSJF[j] && arr_bt.get(j) < min && !arr_status_npSJF[j])
{
                                min = arr_bt.get(j);
                                min_index = j;
```

```
                                flag_npSJF = true;
                            }
                        }
                        if (flag_npSJF) {
                            arr_st_npsjf.add(i);
                            arr_et_npsjf.add(i + arr_bt.get(min_index));
                            parr_npsjf.add(parr.get(min_index));
                            arr_at_npsjf.add(arr_at.get(min_index));
                            i += arr_bt.get(min_index);
                            arr_status_npSJF[min_index] = true;
                        } else {
                            i++;
                        }
                    }
                    int j = 0;
                    int tot_wt = 0;
                    int tot_tat = 0;
                    int delay = 0;
                    for (int i = 0; i < arr_st_npsjf.size(); ++i) {
                        if (delay < arr_st_npsjf.get(i)) {
                            arr_waitstart_npsjf.add(j, delay);
                            delay = arr_st_npsjf.get(i);
                            arr_waitend_npsjf.add(j, delay);
                            j++;
                        }
                        arr_tat_npsjf.add(i, (arr_et_npsjf.get(i) - arr_at_npsjf.get(i)));
                        tot_tat += arr_tat_npsjf.get(i);
                        arr_wt_npsjf.add(i, arr_tat_npsjf.get(i) - (arr_et_npsjf.get(i) -
arr_st_npsjf.get(i)));
                        tot_wt += arr_wt_npsjf.get(i);
                        delay = arr_et_npsjf.get(i);
                    }
                    avg_tat_npsjf = (tot_tat * 1.0) / cnt_process;
                    avg_wt_npsjf = (tot_wt * 1.0) / cnt_process;
                    tt_npsjf = delay;
                }

                if (b_npPri && flag != -1) {
                    boolean[] arr_av_npPri = new boolean[cnt_process];
                    boolean[] arr_status_npPri = new boolean[cnt_process];
                    boolean flag_npPri;
                    for (int i = 0; arr_st_npPri.size() < cnt_process; ) {
                        flag_npPri = false;
                        for (int j = 0; j < arr_at.size(); ++j) {
                            if (i >= arr_at.get(j))
                                arr_av_npPri[j] = true;
                        }
                        int min_pri = Integer.MAX_VALUE;
                        int min_bt = Integer.MAX_VALUE;
                        int min_index = -1;
                        for (int j = 0; j < cnt_process; ++j) {
                            if (arr_av_npPri[j] && !arr_status_npPri[j])
                                if (arr_pri.get(j) < min_pri || (min_bt > arr_bt.get(j) &&
(arr_pri.get(j) == min_pri))) {
                                    min_bt = arr_bt.get(j);
                                    min_pri = arr_pri.get(j);
                                    min_index = j;
                                    flag_npPri = true;
                                }
                        }
                        if (flag_npPri) {
                            arr_st_npPri.add(i);
                            arr_et_npPri.add(i + arr_bt.get(min_index));
                            parr_npPri.add(parr.get(min_index));
                            arr_at_npPri.add(arr_at.get(min_index));
                            i += arr_bt.get(min_index);
                            arr_status_npPri[min_index] = true;
                        } else {
                            i++;
                        }
                    }
                    int j = 0;
                    int tot_wt = 0;
                    int tot_tat = 0;
                    int delay = 0;
```

```
                    for (int i = 0; i < arr_st_npPri.size(); ++i) {
                        if (delay < arr_st_npPri.get(i)) {
                            arr_waitstart_npPri.add(j, delay);
                            delay = arr_st_npPri.get(i);
                            arr_waitend_npPri.add(j, delay);
                            j++;
                        }
                        arr_tat_npPri.add(i, (arr_et_npPri.get(i) - arr_at_npPri.get(i)));
                        tot_tat += arr_tat_npPri.get(i);
                        arr_wt_npPri.add(i, arr_tat_npPri.get(i) - (arr_et_npPri.get(i) -
arr_st_npPri.get(i)));
                        tot_wt += arr_wt_npPri.get(i);
                        delay = arr_et_npPri.get(i);
                    }
                    avg_tat_npPri = (tot_tat * 1.0) / cnt_process;
                    avg_wt_npPri = (tot_wt * 1.0) / cnt_process;
                    tt_npPri = delay;
                }

                if (b_RR && flag != -1) {
                    for (int i = 0; i < arr_at.size(); ++i) {
                        arr_timeleft_RR.add(i, arr_bt.get(i));
                    }
                    boolean[] arr_av_RR = new boolean[cnt_process];
                    boolean[] arr_status_RR = new boolean[cnt_process];
                    int cnt_flag = 0;
                    int tot_tat = 0;
                    int tot_wt = 0;
                    int i = 0, k;
                    while (cnt_flag < cnt_process) {
                        int wait_flag = 0;

                        for (k = 0; k < arr_at.size(); ++k) {
                            for (int j = 0; j < arr_at.size(); ++j) {
                                if (i >= arr_at.get(j))
                                    arr_av_RR[j] = true;
                            }
                            if (arr_av_RR[k] && !arr_status_RR[k]) {
                                arr_st_RR.add(i);
                                parr_RR.add(parr.get(k));
                                wait_flag++;
                                if (arr_timeleft_RR.get(k) > tq) {
                                    arr_timeleft_RR.set(k, arr_timeleft_RR.get(k) - tq);
                                    arr_et_RR.add(i + tq);
                                    i += tq;
                                } else {
                                    arr_et_RR.add(i + arr_timeleft_RR.get(k));
                                    i += arr_timeleft_RR.get(k);
                                    arr_timeleft_RR.set(k, 0);
                                    arr_status_RR[k] = true;
                                    cnt_flag++;
                                    arr_tat_RR.add(i - arr_at.get(k));
                                    tot_tat += i - arr_at.get(k);
                                    arr_wt_RR.add(i - arr_at.get(k) - arr_bt.get(k));
                                    tot_wt += i - arr_at.get(k) - arr_bt.get(k);
                                }
                            }
                        }
                        if (wait_flag == 0) {
                            arr_waitstart_RR.add(i);
                            ++i;
                            arr_waitend_RR.add(i);
                        }
                    }
                    tt_RR = i;
                    avg_tat_RR = (tot_tat * 1.0) / cnt_process;
                    avg_wt_RR = (tot_wt * 1.0) / cnt_process;
                }

                if (b_preSJF && flag != -1) {
                    int[] arr_ct_preSJF = new int[cnt_process];
                    int[] arr_at_preSJF = new int[cnt_process];
                    for (int i = 0; i < cnt_process; ++i) {
                        arr_at_preSJF[i] = Integer.MAX_VALUE;
                        arr_ct_preSJF[i] = Integer.MIN_VALUE;
```

```
                    }
                    for (int i = 0; i < cnt_process; ++i) {
                        arr_timeleft_preSJF.add(i, arr_bt.get(i));
                    }
                    boolean[] arr_av_preSJF = new boolean[cnt_process];
                    boolean[] arr_status_preSJF = new boolean[cnt_process];
                    boolean flag_preSJF;
                    int cnt_flag = 0;
                    for (int i = 0; cnt_flag < cnt_process; ++i) {
                        flag_preSJF = false;
                        for (int j = 0; j < arr_at.size(); ++j) {
                            if (i >= arr_at.get(j))
                                arr_av_preSJF[j] = true;
                        }
                        int min = Integer.MAX_VALUE;
                        int min_index = -1;
                        for (int j = 0; j < cnt_process; ++j) {
                            if (arr_av_preSJF[j] && arr_timeleft_preSJF.get(j) < min &&
!arr_status_preSJF[j]) {
                                min = arr_timeleft_preSJF.get(j);
                                min_index = j;
                                flag_preSJF = true;
                            }
                        }
                        if (flag_preSJF) {
                            arr_st_preSJF.add(i);
                            arr_et_preSJF.add(i + 1);
                            arr_timeleft_preSJF.set(min_index,
arr_timeleft_preSJF.get(min_index) - 1);
                            parr_preSJF.add(parr.get(min_index));
                            int pos = Integer.parseInt(parr.get(min_index).substring(1)) - 1;
                            if (arr_ct_preSJF[pos] < i + 1)
                                arr_ct_preSJF[pos] = i + 1;
                            arr_at_preSJF[pos] = arr_at.get(pos);
                            if (arr_timeleft_preSJF.get(min_index) == 0) {
                                arr_status_preSJF[min_index] = true;
                                cnt_flag++;
                            }
                        }
                    }
                    int j = 0;
                    int tot_wt = 0;
                    int tot_tat = 0;
                    int delay = 0;
                    for (int i = 0; i < arr_st_preSJF.size(); ++i) {
                        if (delay < arr_st_preSJF.get(i)) {
                            arr_waitstart_preSJF.add(j, delay);
                            delay++;
                            arr_waitend_preSJF.add(j, delay);
                            j++;
                        }
                    }
                    for (int i = 0; i < arr_ct_preSJF.length; ++i) {
                        if (delay < arr_ct_preSJF[i])
                            delay = arr_ct_preSJF[i];
                        arr_tat_preSJF.add(i, arr_ct_preSJF[i] - arr_at_preSJF[i]);
                        tot_tat += arr_ct_preSJF[i] - arr_at_preSJF[i];
                        arr_wt_preSJF.add(i, arr_tat_preSJF.get(i) - arr_bt.get(i));
                        tot_wt += arr_tat_preSJF.get(i) - arr_bt.get(i);
                    }
                    avg_tat_preSJF = (tot_tat * 1.0) / cnt_process;
                    avg_wt_preSJF = (tot_wt * 1.0) / cnt_process;
                    tt_preSJF = delay;
                }

                if (b_prePri && flag != -1) {
                    int[] arr_ct_prePri = new int[cnt_process];
                    int[] arr_at_prePri = new int[cnt_process];
                    for (int i = 0; i < cnt_process; ++i) {
                        arr_at_prePri[i] = Integer.MAX_VALUE;
                        arr_ct_prePri[i] = Integer.MIN_VALUE;
                    }
                    for (int i = 0; i < cnt_process; ++i) {
                        arr_timeleft_prePri.add(i, arr_bt.get(i));
                    }
```

```
                    boolean[] arr_av_prePri = new boolean[cnt_process];
                    boolean[] arr_status_prePri = new boolean[cnt_process];
                    boolean flag_prePri;
                    int cnt_flag = 0;
                    for (int i = 0; cnt_flag < cnt_process; ++i) {
                        flag_prePri = false;
                        for (int j = 0; j < arr_at.size(); ++j) {
                            if (i >= arr_at.get(j))
                                arr_av_prePri[j] = true;
                        }
                        int min = Integer.MAX_VALUE;
                        int min_pri = Integer.MAX_VALUE;
                        int min_index = -1;
                        for (int j = 0; j < cnt_process; ++j) {
                            if (arr_av_prePri[j] && !arr_status_prePri[j])
                                if (arr_pri.get(j) < min_pri || (min >
arr_timeleft_prePri.get(j) && (arr_pri.get(j) == min_pri))) {
                                    min = arr_timeleft_prePri.get(j);
                                    min_index = j;
                                    min_pri=arr_pri.get(j);
                                    flag_prePri = true;
                                }
                        }
                        if (flag_prePri) {
                            arr_st_prePri.add(i);
                            arr_et_prePri.add(i + 1);
                            arr_timeleft_prePri.set(min_index,
arr_timeleft_prePri.get(min_index) - 1);
                            parr_prePri.add(parr.get(min_index));
                            int pos = Integer.parseInt(parr.get(min_index).substring(1)) - 1;
                            if (arr_ct_prePri[pos] < i + 1)
                                arr_ct_prePri[pos] = i + 1;
                            arr_at_prePri[pos] = arr_at.get(pos);
                            if (arr_timeleft_prePri.get(min_index) == 0) {
                                arr_status_prePri[min_index] = true;
                                cnt_flag++;
                            }
                        }
                    }
                    int j = 0;
                    int tot_wt = 0;
                    int tot_tat = 0;
                    int delay = 0;
                    for (int i = 0; i < arr_st_prePri.size(); ++i) {
                        if (delay < arr_st_prePri.get(i)) {
                            arr_waitstart_prePri.add(j, delay);
                            delay++;
                            arr_waitend_prePri.add(j, delay);
                            j++;
                        }
                    }
                    for (int i = 0; i < arr_ct_prePri.length; ++i) {
                        if (delay < arr_ct_prePri[i])
                            delay = arr_ct_prePri[i];
                        arr_tat_prePri.add(i, arr_ct_prePri[i] - arr_at_prePri[i]);
                        tot_tat += arr_ct_prePri[i] - arr_at_prePri[i];
                        arr_wt_prePri.add(i, arr_tat_prePri.get(i) - arr_bt.get(i));
                        tot_wt += arr_tat_prePri.get(i) - arr_bt.get(i);
                    }
                    avg_tat_prePri = (tot_tat * 1.0) / cnt_process;
                    avg_wt_prePri = (tot_wt * 1.0) / cnt_process;
                    tt_prePri = delay;
                }

                if (flag != -1) {
                    ganttframe.setSize(1025, 800);
                    ganttframe.setLocationRelativeTo(null);
                    ganttframe.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
                    ganttframe.setTitle("Gantt Charts");
                    ganttframe.add(canvas);
                    ganttframe.setResizable(false);
                    repaint();
                    ganttframe.setVisible(true);
                }
            }
```

```
        });
    }

    public static class Canvas extends JPanel {
        public void paintComponent(Graphics g) {
            super.paintComponent(g);

            //FCFS
            g.setFont(new Font("Arial", Font.PLAIN, 16));
            g.drawString("First Come First Served", 420, 110);
            g.drawLine(0, 120, 1025, 120);
            if (b_FCFS) {
                ArrayList<Integer> times_percFCFS = new ArrayList<>();
                for (int i = 0; i < cnt_process; ++i) {
                    times_percFCFS.add(i, (int) (arr_bt.get(i) * 1000.0 / tt_fcfs));
                }
                int cnt = 0;
                int sum = 0;
                int j = 0;
                for (int i = 0; i < times_percFCFS.size(); ++i) {
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    if (sum < arr_at.get(i)) {
                        g.setColor(new Color(255, 255, 255));
                        g.fillRect(cnt, 0, ((int) ((arr_waitend_fcfs.get(j) -
arr_waitstart_fcfs.get(j)) * 1000.0 / tt_fcfs)), 65);
                        g.setColor(Color.BLACK);
                        g.setFont(new Font("Arial", Font.PLAIN, 14));
                        g.drawString(Integer.toString(arr_waitend_fcfs.get(j)), cnt + ((int)
((arr_waitend_fcfs.get(j) - arr_waitstart_fcfs.get(j)) * 1000.0 / tt_fcfs)) - 25, 60);
                        g.setFont(new Font("Arial", Font.ITALIC, 16));
                        g.drawString("Idle", cnt + ((int) ((arr_waitend_fcfs.get(j) -
arr_waitstart_fcfs.get(j)) * 1000.0 / tt_fcfs)) / 2 - 10, 40);
                        cnt += ((int) ((arr_waitend_fcfs.get(j) - arr_waitstart_fcfs.get(j)) *
1000.0 / tt_fcfs));
                        sum = arr_waitend_fcfs.get(j);
                        j++;
                    }
                    int r = red[Integer.parseInt(parr.get(i).substring(1)) - 1];
                    int gr = green[Integer.parseInt(parr.get(i).substring(1)) - 1];
                    int b = blue[Integer.parseInt(parr.get(i).substring(1)) - 1];
                    Color color = new Color(r, gr, b);
                    g.setColor(color);
                    g.fillRect(cnt, 0, times_percFCFS.get(i), 65);
                    g.setColor(Color.BLACK);
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    g.drawString(Integer.toString(sum + arr_bt.get(i)), cnt +
times_percFCFS.get(i) - 25, 60);
                    g.setFont(new Font("Arial", Font.BOLD, 16));
                    g.drawString(parr.get(i), cnt + (times_percFCFS.get(i)) / 2 - 10, 40);
                    cnt += times_percFCFS.get(i);
                    sum += arr_bt.get(i);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Waiting Times: ", 10, 90);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    //Waiting times
                    int cnt_wt = 110;
                    for (int m = 0; m < parr.size(); ++m) {
                        g.drawString(parr.get(m) + ": ", cnt_wt, 90);
                        g.drawString(arr_wt_fcfs.get(m) + " ns", cnt_wt + 20, 90);
                        cnt_wt += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Waiting Time: " + avg_wt_fcfs + " ns", cnt_wt, 90);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    //Turnaround times
                    g.drawString("Turnaround Times: ", 10, 80);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    int cnt_tat = 110;
                    for (int m = 0; m < parr.size(); ++m) {
                        g.drawString(parr.get(m) + ": ", cnt_tat, 80);
                        g.drawString(arr_tat_fcfs.get(m) + " ns", cnt_tat + 20, 80);
                        cnt_tat += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Turnaround Time: " + avg_tat_fcfs + " ns", cnt_tat, 80);
```

```
                }
            } else {
                g.setColor(Color.GRAY);
                g.setFont(new Font("Arial", Font.ITALIC, 12));
                g.drawString("To view this Gantt chart, check the box which says FCFS.", 100,
50);
            }

            //RR
            g.setColor(Color.BLACK);
            g.setFont(new Font("Arial", Font.PLAIN, 16));
            g.drawString("Round Robin", 455, 240);
            g.drawLine(0, 250, 1025, 250);
            if (b_RR) {
                ArrayList<Integer> times_percRR = new ArrayList<>();
                for (int i = 0; i < arr_st_RR.size(); ++i) {
                    times_percRR.add(i, (int) ((arr_et_RR.get(i) - arr_st_RR.get(i)) * 1000.0 /
tt_RR));
                }
                int cnt = 0;
                int sum = 0;
                int j = 0;
                for (int i = 0; i < arr_st_RR.size(); ) {
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    while (sum < arr_st_RR.get(i)) {
                        g.setColor(new Color(255, 255, 255));
                        g.fillRect(cnt, 130, ((int) ((arr_waitend_RR.get(j) -
arr_waitstart_RR.get(j)) * 1000.0 / tt_RR)), 65);
                        g.setColor(Color.BLACK);
                        g.setFont(new Font("Arial", Font.PLAIN, 14));
                        g.drawString(Integer.toString(arr_waitend_RR.get(j)), cnt + ((int)
((arr_waitend_RR.get(j) - arr_waitstart_RR.get(j)) * 1000.0 / tt_RR)) - 25, 450 - 260);
                        g.setFont(new Font("Arial", Font.ITALIC, 16));
                        g.drawString("Idle", cnt + ((int) ((arr_waitend_RR.get(j) -
arr_waitstart_RR.get(j)) * 1000.0 / tt_RR)) / 2 - 10, 430 - 260);
                        cnt += ((int) ((arr_waitend_RR.get(j) - arr_waitstart_RR.get(j)) *
1000.0 / tt_RR));
                        sum = arr_waitend_RR.get(j);
                        j++;
                    }
                    int r = red[Integer.parseInt(parr_RR.get(i).substring(1)) - 1];
                    int gr = green[Integer.parseInt(parr_RR.get(i).substring(1)) - 1];
                    int b = blue[Integer.parseInt(parr_RR.get(i).substring(1)) - 1];
                    Color color = new Color(r, gr, b);
                    g.setColor(color);
                    g.fillRect(cnt, 390 - 260, times_percRR.get(i), 65);
                    g.setColor(Color.BLACK);
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    g.drawString(Integer.toString(arr_et_RR.get(i)), cnt + times_percRR.get(i) -
25, 450 - 260);
                    g.setFont(new Font("Arial", Font.BOLD, 16));
                    g.drawString(parr_RR.get(i), cnt + (times_percRR.get(i)) / 2 - 10, 430 -
260);
                    cnt += times_percRR.get(i);
                    sum = arr_et_RR.get(i);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Waiting Times: ", 10, 480 - 260);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    //Waiting times
                    int cnt_wt = 110;
                    for (int m = 0; m < arr_tat_RR.size(); ++m) {
                        g.drawString(parr_RR.get(m) + ": ", cnt_wt, 480 - 260);
                        g.drawString(arr_wt_RR.get(m) + " ns", cnt_wt + 20, 480 - 260);
                        cnt_wt += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Waiting Time: " + avg_wt_RR + " ns", cnt_wt, 480 - 260);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    //Turnaround times
                    g.drawString("Turnaround Times: ", 10, 470 - 260);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    int cnt_tat = 110;
                    for (int m = 0; m < arr_tat_RR.size(); ++m) {
                        g.drawString(parr_RR.get(m) + ": ", cnt_tat, 470 - 260);
                        g.drawString(arr_tat_RR.get(m) + " ns", cnt_tat + 20, 470 - 260);
```

```
                        cnt_tat += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Turnaround Time: " + avg_tat_RR + " ns", cnt_tat, 470 -
260);
                    ++i;
                }
            } else {
                g.setColor(Color.GRAY);
                g.setFont(new Font("Arial", Font.ITALIC, 12));
                g.drawString("To view this Gantt chart, check the box which says Round Robin.",
100, 180);
            }
            //PreSJF
            g.setColor(Color.BLACK);
            g.setFont(new Font("Arial", Font.PLAIN, 16));
            g.drawString("Preemptive SJF", 445, 370);
            g.drawLine(0, 380, 1025, 380);
            if (b_preSJF) {
                ArrayList<Integer> times_percSJF = new ArrayList<>();
                for (int i = 0; i < arr_st_preSJF.size(); ++i) {
                    times_percSJF.add(i, (int) ((arr_et_preSJF.get(i) - arr_st_preSJF.get(i)) *
1000.0 / tt_preSJF));
                }
                int cnt = 0;
                int sum = 0;
                int j = 0;
                for (int i = 0; i < arr_st_preSJF.size(); ++i) {
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    if (sum < arr_st_preSJF.get(i)) {
                        g.setColor(new Color(255, 255, 255));
                        g.fillRect(cnt, 390 - 130, ((int) ((arr_waitend_preSJF.get(j) -
arr_waitstart_preSJF.get(j)) * 1000.0 / tt_preSJF)), 65);
                        g.setColor(Color.BLACK);
                        g.setFont(new Font("Arial", Font.PLAIN, 14));
                        g.drawString(Integer.toString(arr_waitend_preSJF.get(j)), cnt + ((int)
((arr_waitend_preSJF.get(j) - arr_waitstart_preSJF.get(j)) * 1000.0 / tt_preSJF)) - 25, 450 -
130);
                        g.setFont(new Font("Arial", Font.ITALIC, 16));
                        g.drawString("Idle", cnt + ((int) ((arr_waitend_preSJF.get(j) -
arr_waitstart_preSJF.get(j)) * 1000.0 / tt_preSJF)) / 2 - 10, 430 - 130);
                        cnt += ((int) ((arr_waitend_preSJF.get(j) - arr_waitstart_preSJF.get(j))
* 1000.0 / tt_preSJF));
                        sum = arr_waitend_preSJF.get(j);
                        j++;
                    }
                    int r = red[Integer.parseInt(parr_preSJF.get(i).substring(1)) - 1];
                    int gr = green[Integer.parseInt(parr_preSJF.get(i).substring(1)) - 1];
                    int b = blue[Integer.parseInt(parr_preSJF.get(i).substring(1)) - 1];
                    Color color = new Color(r, gr, b);
                    g.setColor(color);
                    g.fillRect(cnt, 390 - 130, times_percSJF.get(i), 65);
                    g.setColor(Color.BLACK);
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    g.drawString(Integer.toString(arr_et_preSJF.get(i)), cnt +
times_percSJF.get(i) - 25, 450 - 130);
                    g.setFont(new Font("Arial", Font.BOLD, 16));
                    g.drawString(parr_preSJF.get(i), cnt + (times_percSJF.get(i)) / 2 - 10, 430
- 130);
                    cnt += times_percSJF.get(i);
                    sum = arr_et_preSJF.get(i);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Waiting Times: ", 10, 480 - 130);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    //Waiting times
                    int cnt_wt = 110;
                    for (int m = 0; m < parr.size(); ++m) {
                        g.drawString(parr_preSJF.get(m) + ": ", cnt_wt, 480 - 130);
                        g.drawString(arr_wt_preSJF.get(m) + " ns", cnt_wt + 20, 480 - 130);
                        cnt_wt += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Waiting Time: " + avg_wt_preSJF + " ns", cnt_wt, 480 -
130);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
```

```
                    //Turnaround times
                    g.drawString("Turnaround Times: ", 10, 470 - 130);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    int cnt_tat = 110;
                    for (int m = 0; m < parr.size(); ++m) {
                        g.drawString(parr_preSJF.get(m) + ": ", cnt_tat, 470 - 130);
                        g.drawString(arr_tat_preSJF.get(m) + " ns", cnt_tat + 20, 470 - 130);
                        cnt_tat += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Turnaround Time: " + avg_tat_preSJF + " ns", cnt_tat, 470
- 130);
                }
            } else {
                g.setColor(Color.GRAY);
                g.setFont(new Font("Arial", Font.ITALIC, 12));
                g.drawString("To view this Gantt chart, check the box which says Preemptive
SJF.", 100, 310);
            }

            //NP SJF
            g.setColor(Color.BLACK);
            g.setFont(new Font("Arial", Font.PLAIN, 16));
            g.drawString("Non-Preemptive SJF", 430, 500);
            g.drawLine(0, 510, 1025, 510);
            if (b_npSJF) {
                ArrayList<Integer> times_percSJF = new ArrayList<>();
                for (int i = 0; i < cnt_process; ++i) {
                    times_percSJF.add(i, (int) ((arr_et_npsjf.get(i) - arr_st_npsjf.get(i)) *
1000.0 / tt_npsjf));
                }
                int cnt = 0;
                int sum = 0;
                int j = 0;
                for (int i = 0; i < times_percSJF.size(); ++i) {
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    if (sum < arr_st_npsjf.get(i)) {
                        g.setColor(new Color(255, 255, 255));
                        g.fillRect(cnt, 390, ((int) ((arr_waitend_npsjf.get(j) -
arr_waitstart_npsjf.get(j)) * 1000.0 / tt_npsjf)), 65);
                        g.setColor(Color.BLACK);
                        g.setFont(new Font("Arial", Font.PLAIN, 14));
                        g.drawString(Integer.toString(arr_waitend_npsjf.get(j)), cnt + ((int)
((arr_waitend_npsjf.get(j) - arr_waitstart_npsjf.get(j)) * 1000.0 / tt_npsjf)) - 25, 450);
                        g.setFont(new Font("Arial", Font.ITALIC, 16));
                        g.drawString("Idle", cnt + ((int) ((arr_waitend_npsjf.get(j) -
arr_waitstart_npsjf.get(j)) * 1000.0 / tt_npsjf)) / 2 - 10, 430);
                        cnt += ((int) ((arr_waitend_npsjf.get(j) - arr_waitstart_npsjf.get(j)) *
1000.0 / tt_npsjf));
                        sum = arr_waitend_npsjf.get(j);
                        j++;
                    }
                    int r = red[Integer.parseInt(parr_npsjf.get(i).substring(1)) - 1];
                    int gr = green[Integer.parseInt(parr_npsjf.get(i).substring(1)) - 1];
                    int b = blue[Integer.parseInt(parr_npsjf.get(i).substring(1)) - 1];
                    Color color = new Color(r, gr, b);
                    g.setColor(color);
                    g.fillRect(cnt, 390, times_percSJF.get(i), 65);
                    g.setColor(Color.BLACK);
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    g.drawString(Integer.toString(arr_et_npsjf.get(i)), cnt +
times_percSJF.get(i) - 25, 450);
                    g.setFont(new Font("Arial", Font.BOLD, 16));
                    g.drawString(parr_npsjf.get(i), cnt + (times_percSJF.get(i)) / 2 - 10, 430);
                    cnt += times_percSJF.get(i);
                    sum = arr_et_npsjf.get(i);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Waiting Times: ", 10, 480);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    //Waiting times
                    int cnt_wt = 110;
                    for (int m = 0; m < parr_npsjf.size(); ++m) {
                        g.drawString(parr_npsjf.get(m) + ": ", cnt_wt, 480);
                        g.drawString(arr_wt_npsjf.get(m) + " ns", cnt_wt + 20, 480);
                        cnt_wt += 50;
```

```
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Waiting Time: " + avg_wt_npsjf + " ns", cnt_wt, 480);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    //Turnaround times
                    g.drawString("Turnaround Times: ", 10, 470);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    int cnt_tat = 110;
                    for (int m = 0; m < parr_npsjf.size(); ++m) {
                        g.drawString(parr_npsjf.get(m) + ": ", cnt_tat, 470);
                        g.drawString(arr_tat_npsjf.get(m) + " ns", cnt_tat + 20, 470);
                        cnt_tat += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Turnaround Time: " + avg_tat_npsjf + " ns", cnt_tat,
470);
                }
            } else {
                g.setColor(Color.GRAY);
                g.setFont(new Font("Arial", Font.ITALIC, 12));
                g.drawString("To view this Gantt chart, check the box which says Non-Preemptive
SJF.", 100, 440);
            }

            //Preemptive Priority
            g.setColor(Color.BLACK);
            g.setFont(new Font("Arial", Font.PLAIN, 16));
            g.drawString("Preemptive Priority", 430, 630);
            g.drawLine(0, 640, 1025, 640);
            if (b_prePri) {
                ArrayList<Integer> times_percSJF = new ArrayList<>();
                for (int i = 0; i < arr_st_prePri.size(); ++i) {
                    times_percSJF.add(i, (int) ((arr_et_prePri.get(i) - arr_st_prePri.get(i)) *
1000.0 / tt_prePri));
                }
                int cnt = 0;
                int sum = 0;
                int j = 0;
                for (int i = 0; i < arr_st_prePri.size(); ++i) {
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    if (sum < arr_st_prePri.get(i)) {
                        g.setColor(new Color(255, 255, 255));
                        g.fillRect(cnt, 390 + 130, ((int) ((arr_waitend_prePri.get(j) -
arr_waitstart_prePri.get(j)) * 1000.0 / tt_prePri)), 65);
                        g.setColor(Color.BLACK);
                        g.setFont(new Font("Arial", Font.PLAIN, 14));
                        g.drawString(Integer.toString(arr_waitend_prePri.get(j)), cnt + ((int)
((arr_waitend_prePri.get(j) - arr_waitstart_prePri.get(j)) * 1000.0 / tt_prePri)) - 25, 450 +
130);
                        g.setFont(new Font("Arial", Font.ITALIC, 16));
                        g.drawString("Idle", cnt + ((int) ((arr_waitend_prePri.get(j) -
arr_waitstart_prePri.get(j)) * 1000.0 / tt_prePri)) / 2 - 10, 430 + 130);
                        cnt += ((int) ((arr_waitend_prePri.get(j) - arr_waitstart_prePri.get(j))
* 1000.0 / tt_prePri));
                        sum = arr_waitend_prePri.get(j);
                        j++;
                    }
                    int r = red[Integer.parseInt(parr_prePri.get(i).substring(1)) - 1];
                    int gr = green[Integer.parseInt(parr_prePri.get(i).substring(1)) - 1];
                    int b = blue[Integer.parseInt(parr_prePri.get(i).substring(1)) - 1];
                    Color color = new Color(r, gr, b);
                    g.setColor(color);
                    g.fillRect(cnt, 390 + 130, times_percSJF.get(i), 65);
                    g.setColor(Color.BLACK);
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    g.drawString(Integer.toString(arr_et_prePri.get(i)), cnt +
times_percSJF.get(i) - 25, 450 + 130);
                    g.setFont(new Font("Arial", Font.BOLD, 16));
                    g.drawString(parr_prePri.get(i), cnt + (times_percSJF.get(i)) / 2 - 10, 430
+ 130);
                    cnt += times_percSJF.get(i);
                    sum = arr_et_prePri.get(i);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Waiting Times: ", 10, 480 + 130);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
```

```
                    //Waiting times
                    int cnt_wt = 110;
                    for (int m = 0; m < parr.size(); ++m) {
                        g.drawString(parr_prePri.get(m) + ": ", cnt_wt, 480 + 130);
                        g.drawString(arr_wt_prePri.get(m) + " ns", cnt_wt + 20, 480 + 130);
                        cnt_wt += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Waiting Time: " + avg_wt_prePri + " ns", cnt_wt, 480 +
130);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    //Turnaround times
                    g.drawString("Turnaround Times: ", 10, 470 + 130);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    int cnt_tat = 110;
                    for (int m = 0; m < parr.size(); ++m) {
                        g.drawString(parr_prePri.get(m) + ": ", cnt_tat, 470 + 130);
                        g.drawString(arr_tat_prePri.get(m) + " ns", cnt_tat + 20, 470 + 130);
                        cnt_tat += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Turnaround Time: " + avg_tat_prePri + " ns", cnt_tat, 470
+ 130);
                }
            } else {
                g.setColor(Color.GRAY);
                g.setFont(new Font("Arial", Font.ITALIC, 12));
                g.drawString("To view this Gantt chart, check the box which says Preemptive
Priority.", 100, 570);
            }

            //Non-Preemptive Priority
            g.setColor(Color.BLACK);
            g.setFont(new Font("Arial", Font.PLAIN, 16));
            g.drawString("Non-Preemptive Priority", 415, 760);
            if (b_npPri) {
                ArrayList<Integer> times_percnpPri = new ArrayList<>();
                for (int i = 0; i < cnt_process; ++i) {
                    times_percnpPri.add(i, (int) ((arr_et_npPri.get(i) - arr_st_npPri.get(i)) *
1000.0 / tt_npPri));
                }
                int cnt = 0;
                int sum = 0;
                int j = 0;
                for (int i = 0; i < times_percnpPri.size(); ++i) {
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    if (sum < arr_st_npPri.get(i)) {
                        g.setColor(new Color(255, 255, 255));
                        g.fillRect(cnt, 650, ((int) ((arr_waitend_npPri.get(j) -
arr_waitstart_npPri.get(j)) * 1000.0 / tt_npPri)), 65);
                        g.setColor(Color.BLACK);
                        g.setFont(new Font("Arial", Font.PLAIN, 14));
                        g.drawString(Integer.toString(arr_waitend_npPri.get(j)), cnt + ((int)
((arr_waitend_npPri.get(j) - arr_waitstart_npPri.get(j)) * 1000.0 / tt_npPri)) - 25, 450 + 260);
                        g.setFont(new Font("Arial", Font.ITALIC, 16));
                        g.drawString("Idle", cnt + ((int) ((arr_waitend_npPri.get(j) -
arr_waitstart_npPri.get(j)) * 1000.0 / tt_npPri)) / 2 - 10, 430 + 260);
                        cnt += ((int) ((arr_waitend_npPri.get(j) - arr_waitstart_npPri.get(j)) *
1000.0 / tt_npPri));
                        sum = arr_waitend_npPri.get(j);
                        j++;
                    }
                    int r = red[Integer.parseInt(parr_npPri.get(i).substring(1)) - 1];
                    int gr = green[Integer.parseInt(parr_npPri.get(i).substring(1)) - 1];
                    int b = blue[Integer.parseInt(parr_npPri.get(i).substring(1)) - 1];
                    Color color = new Color(r, gr, b);
                    g.setColor(color);
                    g.fillRect(cnt, 390 + 260, times_percnpPri.get(i), 65);
                    g.setColor(Color.BLACK);
                    g.setFont(new Font("Arial", Font.PLAIN, 14));
                    g.drawString(Integer.toString(arr_et_npPri.get(i)), cnt +
times_percnpPri.get(i) - 25, 450 + 260);
                    g.setFont(new Font("Arial", Font.BOLD, 16));
                    g.drawString(parr_npPri.get(i), cnt + (times_percnpPri.get(i)) / 2 - 10, 430
+ 260);
```

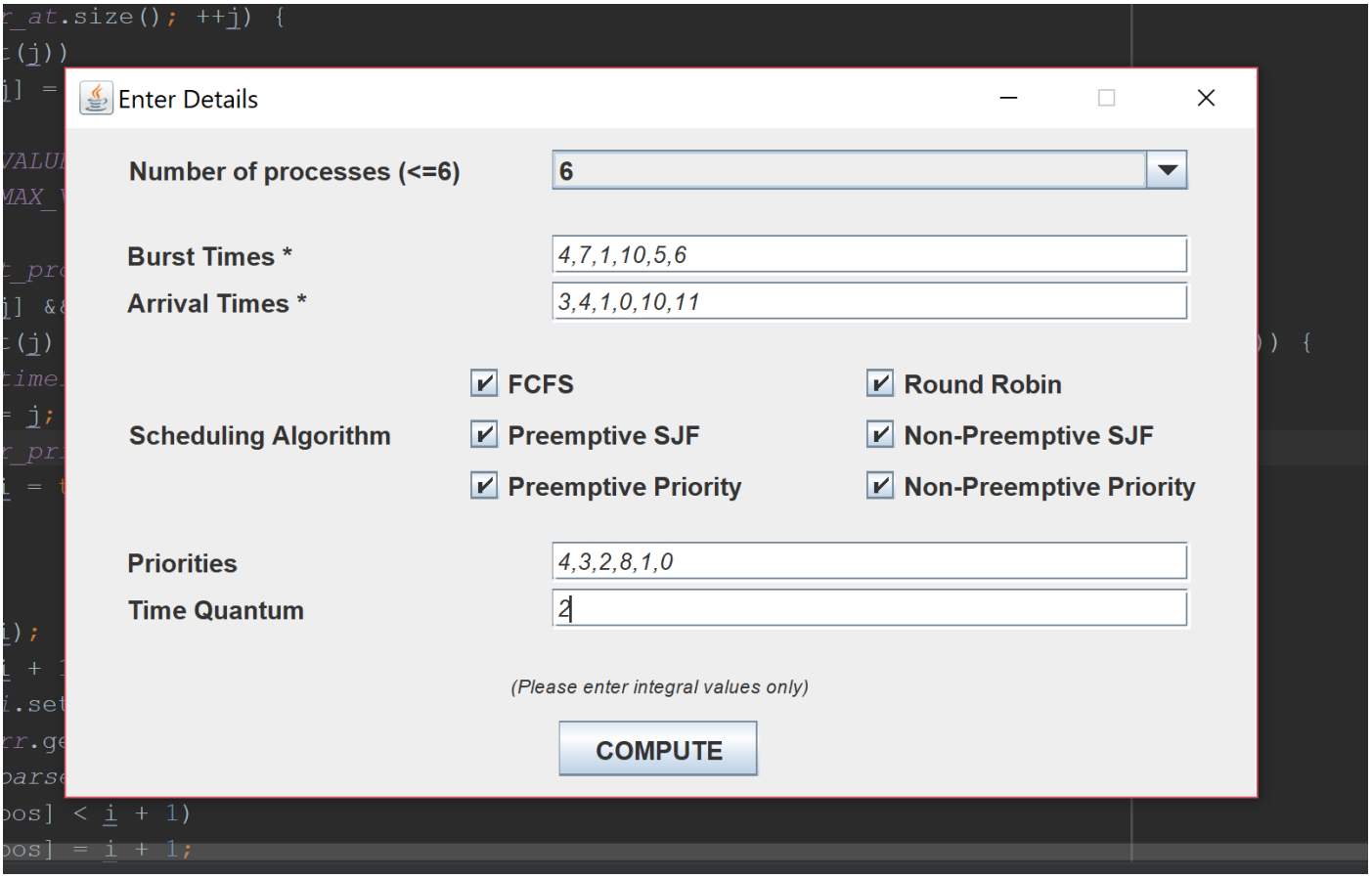
```
                    cnt += times_percnpPri.get(i);
                    sum = arr_et_npPri.get(i);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Waiting Times: ", 10, 480 + 260);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    //Waiting times
                    int cnt_wt = 110;
                    for (int m = 0; m < parr_npPri.size(); ++m) {
                        g.drawString(parr_npPri.get(m) + ": ", cnt_wt, 480 + 260);
                        g.drawString(arr_wt_npPri.get(m) + " ns", cnt_wt + 20, 480 + 260);
                        cnt_wt += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Waiting Time: " + avg_wt_npPri + " ns", cnt_wt, 480 +
260);
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    //Turnaround times
                    g.drawString("Turnaround Times: ", 10, 470 + 260);
                    g.setFont(new Font("Arial", Font.PLAIN, 10));
                    int cnt_tat = 110;
                    for (int m = 0; m < parr_npPri.size(); ++m) {
                        g.drawString(parr_npPri.get(m) + ": ", cnt_tat, 470 + 260);
                        g.drawString(arr_tat_npPri.get(m) + " ns", cnt_tat + 20, 470 + 260);
                        cnt_tat += 50;
                    }
                    g.setFont(new Font("Arial", Font.BOLD, 10));
                    g.drawString("Avg. Turnaround Time: " + avg_tat_npPri + " ns", cnt_tat, 470
+ 260);
                }
            } else {
                g.setColor(Color.GRAY);
                g.setFont(new Font("Arial", Font.ITALIC, 12));
                g.drawString("To view this Gantt chart, check the box which says Non-Preemptive
Priority.", 100, 700);
            }
        }
    }

    public static void main(String[] args) {
        JFrame mainframe = new cpuscheduling();
        mainframe.setSize(500, 500);
        mainframe.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        mainframe.setLocationRelativeTo(null);
        mainframe.pack();
        mainframe.setTitle("Enter Details");
        mainframe.setResizable(false);
        mainframe.setVisible(true);
    }
}
```

Operating Systems (CSD204): Graded Lab Assignment 1

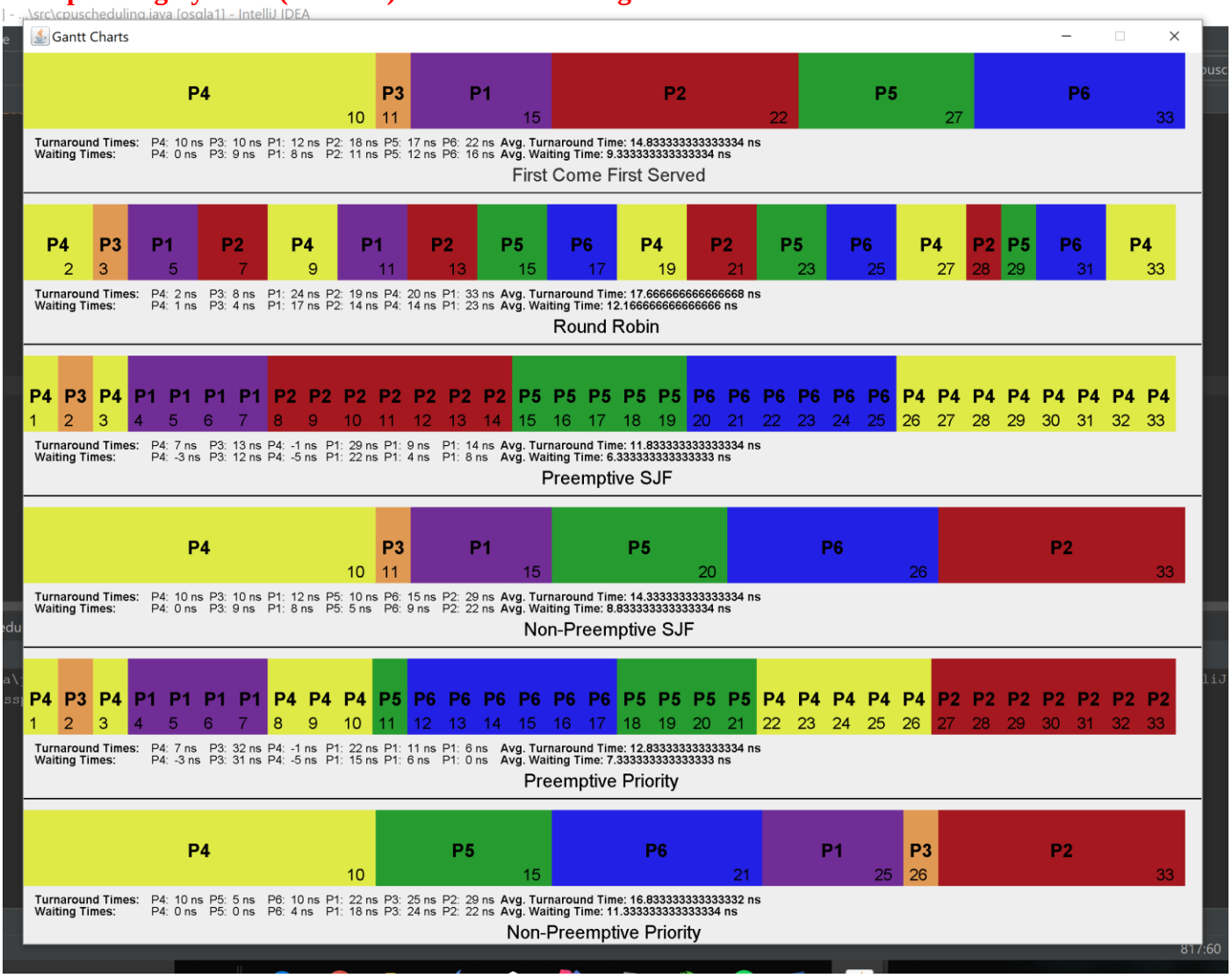
Screenshots

Input Screen

//paste here



Output Screen

//paste here

Operating Systems (CSD204): Graded Lab Assignment 1